

✓ AI Music Generation Project Plan

This project aims to build an AI model capable of generating original MIDI music. We will proceed through the following stages:

1. **Data Collection:** Gather a diverse dataset of MIDI music (e.g., classical, jazz).
2. **Data Preprocessing:** Transform MIDI files into a format suitable for neural networks, such as note sequences, using libraries like `music21`.
3. **Model Building:** Design and implement a deep learning model (e.g., LSTM-based RNN or a GAN) to learn musical patterns and structures.
4. **Model Training:** Train the model on the preprocessed dataset.
5. **Music Generation & Output:** Use the trained model to generate new musical sequences, convert them back into MIDI format, and provide options for playback or saving as audio.

✓ Step 1: Data Collection

The first step is to acquire a suitable dataset of MIDI music. Here, we'll set up a directory to store the data and discuss common sources for MIDI files.

```
# Create a directory to store the MIDI files
import os

data_dir = 'midi_data'
if not os.path.exists(data_dir):
    os.makedirs(data_dir)
    print(f"Created directory: {data_dir}")
else:
    print(f"Directory already exists: {data_dir}")
```

✓ Sources for MIDI Music Datasets

There are several places where you can find MIDI datasets for training:

- **Kaggle:** Many users upload curated MIDI datasets. You can search for terms like "MIDI classical music" or "jazz MIDI dataset" (e.g., [Maestro Dataset](#)). You'll typically need to download these manually or use the Kaggle API.
- **Google Magenta:** Magenta provides several datasets specifically designed for music generation research, such as the [Lakh MIDI Dataset](#) (LMD).
- **Other Public Repositories:** Various academic institutions or individual researchers might host MIDI datasets.

For Kaggle datasets, you would typically download them to your local machine and then upload them to Colab, or use the Kaggle API (which requires setting up credentials).

Here's an example of how you might download a file if you have a direct public URL for a zip archive containing MIDI files. **Please replace the `midi_download_url` with the actual URL of your chosen dataset.**

```
import requests
import zipfile
import io
import os # Added import os

# !!! IMPORTANT: Replace this with the actual URL of your chosen MIDI dataset !!!
```

```
# Example placeholder URL - this URL is unlikely to work, you need to find a real one.
midi_download_url = "https://example.com/path/to/your_midi_dataset.zip"

# Define data_dir here as well to make the cell self-contained
data_dir = 'midi_data'
if not os.path.exists(data_dir):
    os.makedirs(data_dir)
    print(f"Created directory: {data_dir}")
else:
    print(f"Directory already exists: {data_dir}")

# If you have a direct download link for a zip file:
try:
    print(f"Attempting to download MIDI data from: {midi_download_url}")
    response = requests.get(midi_download_url)
    response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)

    # Unzip the content
    with zipfile.ZipFile(io.BytesIO(response.content)) as zf:
        zf.extractall(data_dir)
    print(f"Successfully downloaded and extracted MIDI data to {data_dir}")

except requests.exceptions.RequestException as e:
    print(f"Error downloading or extracting data: {e}")
    print("Please ensure you have a valid direct download URL for a ZIP file containing MIDI files")
    print("Alternatively, manually download a dataset (e.g., from Kaggle) and upload it to the `midi_data` folder")

# Verify if any MIDI files are present
midi_files = [f for f in os.listdir(data_dir) if f.endswith(('.mid', '.midi'))]
if midi_files:
    print(f"Found {len(midi_files)} MIDI files in '{data_dir}'. Here's a sample:")
    for i, f in enumerate(midi_files[:5]): # Print first 5 files
        print(f"- {f}")
else:
    print(f"No MIDI files found in '{data_dir}'. Please ensure your data is correctly placed or downloaded.")
```

```
Created directory: midi_data
Attempting to download MIDI data from: https://example.com/path/to/your_midi_dataset.zip
Error downloading or extracting data: 404 Client Error: Not Found for url: https://example.com/path/to/your_midi_dataset.zip
Please ensure you have a valid direct download URL for a ZIP file containing MIDI files.
Alternatively, manually download a dataset (e.g., from Kaggle) and upload it to the `midi_data` folder
No MIDI files found in 'midi_data'. Please ensure your data is correctly placed or downloaded.
```

✓ Unzipping Manually Uploaded Files

If you have manually uploaded a `.zip` file containing MIDI data to your Colab environment, you can use the following code to unzip it into the `mid_data` directory. Please make sure to replace `'your_uploaded_midi_dataset.zip'` with the actual filename of your uploaded zip archive.

```
import zipfile
import os

# !!! IMPORTANT: Replace 'your_uploaded_midi_dataset.zip' with the actual name of your uploaded zip file
uploaded_zip_file = 'your_uploaded_midi_dataset.zip'

# Ensure data_dir is defined (it should be from previous cells, but for self-containment)
data_dir = 'midi_data'
if not os.path.exists(data_dir):
    os.makedirs(data_dir)

# Construct the full path to the uploaded zip file
# Assuming the zip file is in the root /content/ directory if uploaded directly
zip_file_path = os.path.join('/content', uploaded_zip_file)

if os.path.exists(zip_file_path):
```

```
try:
    with zipfile.ZipFile(zip_file_path, 'r') as zf:
        zf.extractall(data_dir)
        print(f"Successfully unzipped '{uploaded_zip_file}' to '{data_dir}'")

    # Verify if any MIDI files are present after extraction
    midi_files = [f for f in os.listdir(data_dir) if f.endswith(('.mid', '.midi'))]
    if midi_files:
        print(f"Found {len(midi_files)} MIDI files in '{data_dir}'. Here's a sample:")
        for i, f in enumerate(midi_files[:5]): # Print first 5 files
            print(f"- {f}")
    else:
        print(f"No MIDI files found in '{data_dir}' after extraction. Please check the content")

except zipfile.BadZipFile:
    print(f"Error: '{uploaded_zip_file}' is not a valid zip file.")
except Exception as e:
    print(f"An error occurred during unzipping: {e}")
else:
    print(f"Error: The file '{uploaded_zip_file}' was not found in '/content/'. Please ensure it's
```

Error: The file 'your_uploaded_midi_dataset.zip' was not found in '/content/'. Please ensure it's

Start coding or [generate](#) with AI.